

Re-Diffusion: Modeling Latent Residuals with Diffusion for Time-Series Forecasting

Boning Zhang
College of Computer Science
Zhejiang University
zhangbn@zju.edu.cn

Haishuai Wang*
College of Computer Science
Zhejiang University
haishuai.wang@zju.edu.cn

Zehong Hu
Alibaba Group
Hangzhou, China
zehong.hzh@alibaba-inc.com

Jiajun Wang
Alibaba Group
Hangzhou, China
wj178316@alibaba-inc.com

Hongyi Zhang
Alibaba Group
Hangzhou, China
zhanghongyi.zhy@alibaba-inc.com

Jia Jia
Alibaba Group
Hangzhou, China
jj229618@alibaba-inc.com

Abstract

Generative latent diffusion models (LDMs) have been extensively applied in various fields yet underperform in time-series prediction. Therefore, We propose the **Re-Diffusion** model, a latent diffusion approach that generates backbone residuals specifically tailored for time-series forecasting. The model comprises a variational autoencoder that compresses the residuals between the actual future values and the predictions from the backbone into latent space. It also includes a conditional diffusion generator to forecast the potential distribution of these residuals. Our findings reveal that this latent-space methodology particularly enhances existing backbone predictors, by effectively reducing prediction bias through an advanced estimation of complex error distributions. While previous diffusion-based models tend to struggle with long-term forecasting, Re-Diffusion integrates the strengths of diffusion methods, leading to improvements in long-term predictions. Our experimental results indicate that the Re-Diffusion model achieves a 10% promotion over state-of-art predictors, marking a significant advancement in the field of time-series forecasting.

CCS Concepts

• Computing methodologies → Temporal reasoning.

Keywords

Diffusion Model, Time Series Prediction

ACM Reference Format:

Boning Zhang, Haishuai Wang, Zehong Hu, Jiajun Wang, Hongyi Zhang, and Jia Jia. 2026. Re-Diffusion: Modeling Latent Residuals with Diffusion for Time-Series Forecasting. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792539>

*Corresponding author: Haishuai Wang



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792539>

1 Introduction

Time series data is a collection of sequential data points, representing phenomena across various fields that change over time, such as economic trends[4], traffic flow[7], and power usage etc. By extracting and studying meaningful information, time series forecasting plays a critical role in decision-making for downstream applications. Forecasting models shall detect the historical data represented as $\mathbf{x}_{0:L_1} \in \mathbb{R}^{L_1 \times N}$, where various patterns such as long-term trends, seasonality, cyclicity, and irregularities can be recognized by function $f(\mathbf{x}_{0:L_1})$ to predict future data, represented as $\mathbf{y}_{0:L_2} \in \mathbb{R}^{L_2 \times N}$.

After the application of traditional statistical techniques and deep learning approaches such as MLPs[33], RNNs[15], CNNs[21], and GNNs[10] in the past, the development of transformer models has marked a significant advancement in tackling time series forecasting problems. In these models, each data point reflects the state or value at a specific moment, making transformers inherently suitable for point estimation. Specifically, they can express expected outcomes mathematically as $\mathbb{E}[\mathbf{y}_{0:L_2} | \mathbf{x}_{0:L_1}] = f(\mathbf{x}_{0:L_1})$. The capacity empowers transformer-based models to capture modeling long-term dependencies with sequential data[45]. In recent years, various methods have been developed to leverage the great potential of Transformer models, such as patching[28] and cross-dimension modeling[50] to capture efficient features upon Transformer architectures. While these models demonstrate improved performance compared to traditional Transformer-based approaches, there are still limitations that researchers need to address.

When we look back to those functions $f(\mathbf{x}_{0:L_1})$, they commonly employ an additive noise model to represent the future time series data $\mathbf{y}_{0:L_2}$ as $\mathbf{y}_{0:L_2} = f(\mathbf{x}_{0:L_1}) + \epsilon_0$, where ϵ_0 is assumed to follow a normal distribution $\mathcal{N}(0, \sigma^2)$. However, whether the noise distribution can accurately capture the uncertainty surrounding $\mathbf{y}_{0:L_2}$ given $\mathbf{x}_{0:L_1}$ is ignored. In time series forecasting modeling tasks, the establishment of uncertainty approximation plays a crucial role in shaping final predictions. While prior research frequently incorporates timestamps modeling to enhance the predictors[41, 49], these constructions typically involve linear inputs that vary across different datasets. Moreover, the integration of these plug-in designs necessitates concurrent training with foundational transformer models, compromising overall stability.

Recently, diffusion-based generative models have garnered significant attention due to their exceptional outperformance across various fields[3, 30]. These models operate through a two-step

process: an initial forward process that transforms data into noisy distribution, denoted as $p(x_t|x_0)$, and a reverse process $q(x_0|x_t, c, t)$ to regenerate data from the noise. Conditional diffusion models have made substantial contributions to the reverse process, by improving the predictive accuracy of $\mathbb{E}(x_0|x_t)$. In the context of time-series forecasting, diffusion-based models have achieved significant performance enhancements through the introduction of effective conditional embedding[37, 40] and the integration of time series feature extraction[47]. However, current approaches tend to concentrate on the above two aspects, often neglecting the design of inherent structure. As a result, the potential of diffusion models has yet to be realized in short-term prediction, and their capabilities for long-term prediction remain limited.

To address the aforementioned problems, we propose a generalized framework, **Re-Diffusion** (short for Residual Diffusion Network), to improve the predictive capabilities of time series forecasting by focusing on backbone residuals. Specifically, Re-Diffusion leverages a diffusion process to accurately predict the distribution of errors made by trained predictions, which we call as residuals. To this end, we train a residual autoencoder to compress data into a latent space, and apply diffusion to these latent vectors. Additionally, we incorporate global information by utilizing look-back residual features and timestamps as auxiliary conditional observations for diffusion. Finally, we employ a combiner to merge residual predictions from diffusion with predictions from pre-trained backbones, resulting in a refined final forecast. By fusing the robust prediction of error distribution, Re-Diffusion significantly enhances the predictive performance of mainstream long-term forecasting models. Notably, Re-Diffusion serves as a model-agnostic, plug-and-play solution that can be easily integrated into any time-series forecasting architecture. Our contributions lie in three aspects:

- We critically reflect on the architecture of time-series forecasting models and highlight the fact that the uncertainty distribution between predictions and actual data is underexplored.
- We propose a novel method based on the residuals of trained predictors, which sufficiently connect the compressed residual distribution with generative diffusion predictions to compensate for training errors.
- We conduct extensive experiments with multivariate forecasting datasets and multiple forecasting backbones, demonstrating Re-Diffusion's superior enhancement upon recent state-of-the-art methods for long-term forecasting methods.

2 Related Work

2.1 Latent Diffusion Model

Diffusion models have emerged as a leading approach in a variety of fields over the past years [27, 30, 34], with the Latent Diffusion Model standing out as particularly noteworthy [32]. The model aims to generate a target data distribution $p(x)$, by iteratively denoising a normally distributed variable in the latent space. The operation of diffusion probabilistic models revolves two main processes: the forward process and the reverse process. During the forward process, random noise is added to transform the original data distribution into a Gaussian distribution. The original data, represented by the variable distribution X , is always preprocessed after a variational

autoencoder [19] to yield latent vectors $z_0, z_1, \dots, z_N$. The forward process is characterized by a series of variances β_t and $\alpha_t := 1 - \beta_t$, which are related to discrete time steps.

$$q(z_{1:T} | z_0) = \prod_{t=T}^1 q(z_t | z_{t-1}), \quad (1)$$

$$\text{where } q(z_t | z_{t-1}) \sim \mathcal{N}(\sqrt{1 - \beta_t} z_{t-1}, \beta_t \mathbf{I})$$

Here, β_t is a small positive constant to represent the noise level. After t steps of forward diffusion process, the resulting vectors can be described in a general form as follows:

$$\begin{aligned} z_t &\sim q(z_t | z_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x, (1 - \bar{\alpha}_t) \mathbf{I}), \\ z_t &= \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \epsilon \sim \mathcal{N}(0, \mathbf{I}) \end{aligned} \quad (2)$$

where $\bar{\alpha}_t = \prod_{i=1}^t (1 - \beta_i) \in (0, 1)$, and $z_T \sim \mathcal{N}(0, \mathbf{I})$. As the number of diffusion steps increases from 0 to T , the latent variable z_T approaches a Gaussian variable that is independent of the starting point z_0 .

The reverse process is trained to denoise corruptions, employing a learned Gaussian transition defined as a Markov chain to estimate like:

$$\begin{aligned} p_\theta(z_0) &:= p(z_T) \prod_{t=1}^T p_\theta(z_{t-1} | z_t), \\ p_\theta(z_{t-1} | z_t) &= \mathcal{N}(\mu_\theta(z_t), \sigma_\theta(z_t)) \end{aligned} \quad (3)$$

The noise prediction network ϵ_θ is trained through the mean-squared error between the predicted noise $\epsilon_\theta(z_t)$ and the actual sampled Gaussian noise ϵ_t . Several studies have investigated the adaptation of diffusion models for time series forecasting, either in the original data space[31, 37] or in a compressed latent data space[8]. In light of these research, we introduce modifications to the latent diffusion model to accommodate the characteristics of time series residuals:

$$q(z_{t-1} | z_t, z_0) \propto \mathcal{N}(z_{t-1}; \hat{\alpha}_t z_t + \hat{\gamma}_t z_0, \hat{\beta}_t \mathbf{I}) \quad (4)$$

$$\text{, where } \hat{\alpha}_t = \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}, \hat{\gamma}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t}, \hat{\beta}_t = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}.$$

We adapt to the prediction of z^0 instead of traditional prediction of noise ϵ_θ . This approach demonstrated improved performance in time series prediction tasks, with detailed analysis in Appendix A.

2.2 Time-Series Predictor

Recent advancements in natural language processing (NLP) and computer vision (CV) have led to the development of sophisticated deep learning methods tailored for time series forecasting. Transformer-based[11, 16, 18, 24–26, 43] and diffusion-based[2, 20, 40] models have exhibited remarkable sequence modeling capability and scalability, surpassing traditional CNN[44] and RNN[17, 39] models. While diffusion models excel in short-term predictions due to their algorithm of distribution predictions, they tend to lag behind transformer models in long-term forecasting. Although diffusion models have explored long-term capabilities, such as SSSD[1], which employs the S_4 model[13] to capture long-range features, and mr-Diff[36], which generates time series data at multiple resolutions, they still struggle with long-term forecasting. Meanwhile, it lacks an efficient integration with the strengths of existing forecasting methods. For instance, TMDM[22] primarily utilizes predictive values from transformer-based models, like Informer[51] or Autoformer[46], as conditioning inputs. Inspired by the inverted

design of the iTransformer[26], which captures long-term features effectively, we propose the novel design of Re-Diffusion inner embedding structure. This design enables our model to capture long-term features through an inner transformer architecture, while leveraging the distribution prediction strengths of diffusion models to enhance the capabilities of traditional point prediction-based forecasting methods.

2.3 Conditional Generation

Time-Series forecasting can be seen as conditional generation tasks for the outcomes expressed mathematically as $\mathbb{E}[\mathbf{y}_{0:L_2} \mid \mathbf{x}_{0:L_1}]$. Previous work has researched on the connection between series to predict, such as SutraNets[12], which performs autoregressive predictions among the sub-series, with each sub-series being conditionally predicted based on the values of other sub-series. Diffusion-based models have made remarkable strides in time series forecasting due to their conditional generation framework. Research in diffusion-based models has emphasized the significance of incorporating historical data as effective conditional embeddings to guide the reverse process of generation[1, 20, 23]. Previous work has also explored integrating the transformer predictions as prior knowledge in condition[22]. In contrast, our approach not only integrates the utilization of look-back residual representations but also timestamps, which are captured through attention-based embedding models. We advocate for the automatic selection of reasonable conditions, allowing the structure to adaptively balance specially designed conditions.

3 Method

3.1 Residual Latent Embeddings

Latent Embeddings have been illuminated for efficient focus on semantic bits of the data[29, 32] in CV. Within the autoregressive variational autoencoder(VAEs), these embeddings operate in a highly compressed, discrete Gaussian distribution latent space. A bottleneck operation restricts the latent distribution through random selection and employs KL divergence of the vectors for the constraint to $\mathcal{N}(0, \mathbf{I})$, which are suitable to downstream diffusion structure as Eq. 2.

When addressing time series data, it is essential that the latent space not only abstracts high-frequency, imperceptible details but also encapsulates anomalies resulting from data drift. However, aggressive compression for normal distribution will destroy important information. To mitigate this, KL-penalty (weight the KL term by a factor like 10^{-8}) strategy is always imposed to preserve details of original series data, but weaken the binding of normal standardization, as we certify in the experiments as Figure 3. In contrast to the previous work[8] which focuses on original data represented in latent space, our research aims to identify more effective compression strategies in the latent Gaussian distribution specifically tailored for time series.

Time series forecasting models are typically trained using the Mean Squared Error(MSE) loss to minimize the log-likelihood of prediction, based on the assumption that the prediction errors follow a Gaussian distribution. However, this assumption can skew the estimates due to the inherent characteristics of time series data.

We give the reasonable explanations and corresponding measures as below:

THEOREM 3.1. *Let y_t be real series data, and $\hat{y}_t$ be the prediction from a multivariate series model trained with MSE loss. Then, $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$, where $\epsilon_t = y_t - \hat{y}_t$.*

PROOF. Under the assumption:

$$\epsilon_t \sim \mathcal{N}(0, \sigma^2), \quad \text{i.i.d.}$$

, the conditional probabilistic distribution sets as

$$p(y_t \mid \hat{y}_t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_t - \hat{y}_t)^2}{2\sigma^2}\right)$$

$$\log p(\mathbf{y} \mid \hat{\mathbf{y}}) = \sum_{t=1}^T \log p(y_t \mid \hat{y}_t) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=1}^T (y_t - \hat{y}_t)^2.$$

The optimization denotes as:

$$\mathcal{L}_y = \min_{\hat{\mathbf{y}}} \left[\frac{1}{2\sigma^2} \sum_{t=1}^T (y_t - \hat{y}_t)^2 \right] \approx \min_{\hat{\mathbf{y}}} \mathcal{L}_{\text{MSE}} = \frac{1}{T} \sum_{t=1}^T (y_t - \hat{y}_t)^2$$

By the Lindeberg-Feller CLT[9], when $T \rightarrow \infty$:

$$\frac{1}{\sqrt{T}} \sum_{t=1}^T \epsilon_t \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

□

Under proper model specification and stationary independent noise, residuals of an MSE-trained forecasting model are converged to the Gaussian distribution. Our experiments also indicate that in Figure 3. To ensure the generality and effectiveness of high-capacity latent embeddings, we construct an autoencoder that operates on residuals rather than the original time series data as illustrated in Figure 1.

Generative Latent Autoencoder Considering the distribution drift in time-series data, we employ a symmetric structure to pre-train the autoencoder on a reconstruction task. We implement autoregressive attention-based transformer models symmetrically to manage the compression of original residual data into latent space, as well as the reverse process, addressing Out-Of-Distribution challenges within the sequences. We also adapt the embedding methodology from iTransformer[26], treating the complete residual series as tokens. More precisely, given an time series residual y_{Re}^0 , the encoder $\mathcal{E}$ encodes $y_{Re}^0 \in \mathbb{R}^{N \times L}$ into a latent representation $z = \mathcal{E}(y_{Re}^0)$, and the decoder $\mathcal{D}$ reconstructs the residual from the latent, giving $\hat{y}_{Re}^0 = \mathcal{D}(z) = \mathcal{D}(\mathcal{E}(y_{Re}^0))$, where $z \in \mathbb{R}^{N \times E}$, N denotes the variational number and E denotes the embedding size of encoder.

We also find that the normalization restriction of latent space as D_{KL} is not suitable in the time-series forecasting context, which may undermine the stability and efficacy of reconstruction. Our findings suggest that this limitation can be approximated by the theorem:

THEOREM 3.2. $D_{\text{KL}}(q(z|x) \parallel p(z)) \propto \frac{1}{2} \|z\|^2$ when $p(z) = \mathcal{N}(0, \mathbf{I})$

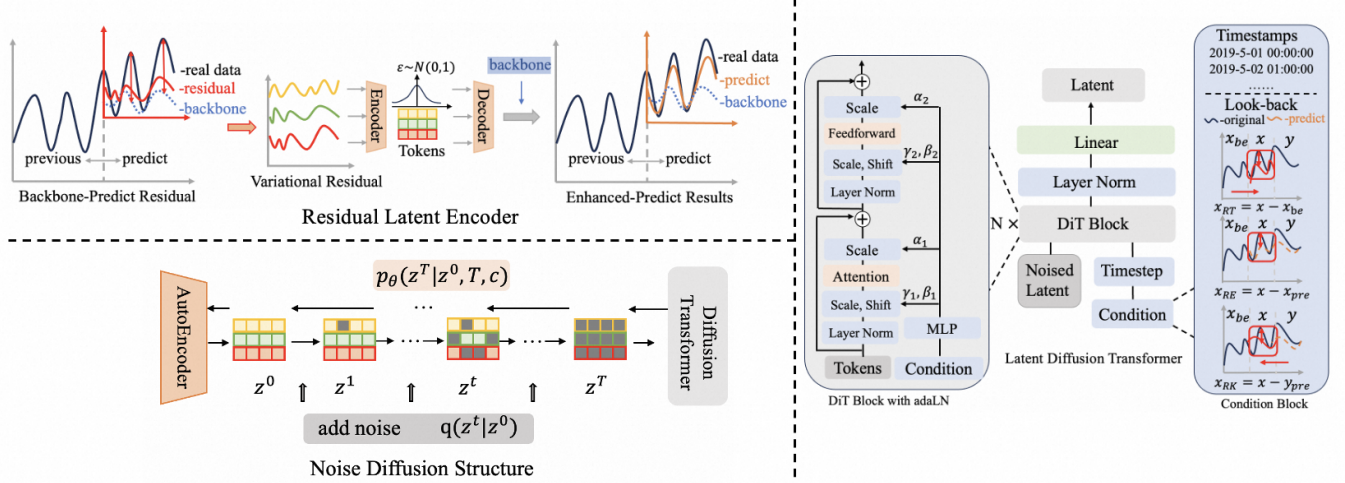


Figure 1: The overview of our proposed Re-Diffusion framework. The left upper part is the residual latent autoencoder model that effectively converts residuals into normalized Gaussian space. The left lower part is the noise diffusion structure of the forward and reverse processes. The right part is the detailed transformer of diffusion, including the DiT Block and Conditional Block to give the conditional generation.

PROOF. Consider a Variational Autoencoder (VAE) where the latent variable z is constrained to follow a standard Gaussian distribution $p(z) = \mathcal{N}(0, I)$. Let the encoder approximate the posterior distribution as $q(z|x) = \mathcal{N}(\mu(x), \Sigma(x))$, where $\Sigma(x) = \text{diag}(\sigma_1^2, \dots, \sigma_d^2)$. The KL divergence term in the VAE loss function is:

$$D_{\text{KL}}(q(z|x) \parallel p(z)) = \mathbb{E}_{q(z|x)} \left[\ln \frac{q(z|x)}{p(z)} \right]. \quad (5)$$

For $q(z|x) = \mathcal{N}(\mu, \Sigma)$ and $p(z) = \mathcal{N}(0, I)$, the KL divergence has a closed-form solution:

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \sum_{i=1}^d (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1), \quad (6)$$

where d is the latent space dimension.

The latent variable z is generated via the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (7)$$

where $\odot$ denotes element-wise multiplication. The expectation of $\|z\|^2$ under $q(z|x)$ is:

$$\begin{aligned} \mathbb{E}_{q(z|x)} [\|z\|^2] &= \mathbb{E} [\|\mu + \sigma \odot \epsilon\|^2] \\ &= \|\mu\|^2 + \mathbb{E} [\|\sigma \odot \epsilon\|^2] \quad (\text{since } \mathbb{E}[\epsilon] = 0) \\ &= \|\mu\|^2 + \sum_{i=1}^d \sigma_i^2 \mathbb{E}[\epsilon_i^2] \\ &= \|\mu\|^2 + \sum_{i=1}^d \sigma_i^2 \quad (\because \mathbb{E}[\epsilon_i^2] = 1) \end{aligned} \quad (8)$$

From (6) and (8), we observe:

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \left(\|\mu\|^2 + \sum_{i=1}^d (\sigma_i^2 - \ln \sigma_i^2 - 1) \right). \quad (9)$$

If the encoder outputs $\sigma_i^2 \approx 1$ (i.e., near-constant variance), the term $\sum_{i=1}^d (\sigma_i^2 - \ln \sigma_i^2 - 1)$ becomes negligible. In this regime:

$$D_{\text{KL}}(q \parallel p) \approx \frac{1}{2} \|\mu\|^2 \propto \mathbb{E}_{q(z|x)} \left[\frac{1}{2} \|z\|^2 \right], \quad (10)$$

where the proportionality follows from (8) under $\sigma_i^2 \approx 1$. $\square$

The theorem proves that minimizing $D_{\text{KL}}(q(z|x) \parallel p(z))$ is equivalent to regularizing $\mathbb{E}[\|z\|^2]$ when the latent variances σ_i^2 are near unity. Therefore, we justify $\frac{1}{2} \|z\|^2$ as a surrogate regularization term in VAEs to enforce Gaussianity of the latent space. The effective constraint with $\frac{1}{2} \|z\|^2$ for the distribution of latent space is expressed as:

$$L_{\text{reg}}(Y_{\text{Re}}; \mathcal{E}, \mathcal{D}) = \min_{\mathcal{E}} \max_{\eta} \left(\frac{1}{2} \|\mathcal{E}(Y_{\text{Re}})\|^2 \right) \quad (11)$$

, distinguishing our approach from the traditional intermediate bottleneck process in standard VAEs[19]. The complete objective training loss function L of our autoencoder is expressed as:

$$L = \min_{\mathcal{E}, \mathcal{D}} \max_{\eta} (L_{\text{rec}}(Y_{\text{Re}}, \mathcal{D}(\mathcal{E}(Y_{\text{Re}}))) + L_{\text{reg}}(Y_{\text{Re}}; \mathcal{E}, \mathcal{D})) \quad (12)$$

where L_{rec} denotes the reconstruction loss, and L_{reg} serves as the regularization loss, guiding the latent space towards a Gaussian distribution.

3.2 Conditioning Generation

Diffusion models can predict the distributions of the form $p(z|c)$ by executing a conditional denoising network over the synthesis process. Unlike previous approaches that primarily focused on look-back windows, we enhance the conditioning process by incorporating more information, such as look-back window residuals and timestamps as illustrated in the right part of Figure 1.

We first derive look-back window residuals from three key aspects: **residual trends**, **residual error** and **residual knowledge**.

We extend the look-back window before the target prediction y^0 . The **residual trends** are calculated as $x_{RT}^0 = x^0 - x_{be}^0$, where x^0 represents the recent look-back window for y^0 and x_{be}^0 stands for the preceding window of x^0 . This adjustment allows us to capture the global changes across the entire timeframe while filtering out cyclical variations. Next, We define the **residual error** as $x_{RE}^0 = x^0 - x_{pre}^0$, where x_{pre}^0 is predicted by the trained backbone predictor using x_{be}^0 as input. Residual error highlights the potential discrepancy in evaluation. Following insights from previous studies, we incorporate the predictor's output as prior **residual knowledge**, denoted as $x_{RK}^0 = x^0 - y_{pre}^0$. We conclude the look-back window residuals as

, where $\{x_{RT}^0, x_{RE}^0, x_{RK}^0\} \in \mathbb{R}^{N \times L}$ construct a comprehensive reservoir of potential residual conditions within the look-back window.

In parallel with such residuals, we also leverage timestamp information s_t . To effectively utilize the temporal data, we employ a domain specific encoder τ_θ that projects $s_t \in \mathbb{R}^{4 \times L}$ to an intermediate representation $\varphi(s_t) = \mathbf{t}^{(\mathcal{T}_k)} \in \mathbb{R}^{N \times L}$. The representation is concatenated with residual data $\{x_{RT}^0, x_{RE}^0, x_{RK}^0\}$ and mapped to intermediate layers via linear encoder.

$$\mathbf{C}^{(\mathcal{T}_k)} = \mathbf{X}_{RT} \parallel \mathbf{X}_{RE} \parallel \mathbf{X}_{RK} \parallel \mathbf{T}^{(\mathcal{T}_k)} \quad (13)$$

into condition $\mathbf{c}^{(\mathcal{T}_k)} \in \mathbb{R}^{N \times 4L}$. Then we map the condition $\mathbf{c}$ to the intermediate layers via linear encoder $\tau_\theta(\mathbf{c}^{(\mathcal{T}_k)}) \in \mathbb{R}^{N \times d_\tau}$ and attention layer:

$$\text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) = \text{softmax}\left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top}{\sqrt{D}}\right) \mathbf{V}_h \quad (14)$$

where queries, keys, and values $\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h \in \mathbb{R}^{N \times d_\tau}$ are learnable projection matrices. The attention mechanism thus provides an effective strategy for drawing features from a diverse range of input information.

Based on the above analysis, we then learn the conditional embedding via $\tau_\theta(\mathbf{c}^{(\mathcal{T}_k)})$, which is more flexibly parameterized with domain-specific experts.

3.3 Diffusion transformer

Diffusion Transformer is designed to learn the data distribution $p(z_0 | c)$ by progressively denoising the normally distributed noise variable illustrated in Figure 1. The process corresponds to learning the reverse dynamics of a fixed Markov Chain of length T as Eq. 3. Previous work in image synthesis focus on predicting a denoised score-matching[38] variant of noisy version input [5, 14, 32, 35], interpreted as a sequence of denoising autoencoders $\epsilon_\theta(x_t, t); t = 1 \dots T$, aimed at estimating noise. The training objective can be expressed as:

$$L_{ND} = \mathbb{E}_{x_t, \epsilon \sim \mathcal{N}(0,1), t} [\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2] \quad (15)$$

where x_t denotes the inputs whether in latent space or original data space. However, in the context of time series predict tasks, the reverse denoising of target noise ϵ is less pronounced compared to z [8]. The corresponding objective in our latent diffusion model is formulated as:

$$L_{DD} := \mathbb{E}_{\mathcal{E}(Y_{Re}), z \sim p(z), c, t} [\|\mathcal{E}(Y_{Re}) - \hat{z}_\theta(z_t, \tau_\theta(c), t)\|_2^2] \quad (16)$$

with t uniformly sampled from $\{1, \dots, T\}$, where $\tau_\theta(c)$ is the conditional embedding as outlined in Sec 3.2. Both $\hat{z}_\theta$ and τ_θ are jointly optimized.

Algorithm 1 Training

Input: Sample $x_{0:T}$ (History) and y^0 (Target) from training set;
Number of diffusion steps T ; Pretrained predictor backbone B ;
Encoder E in pretrained autoencoder

Output: Trained denoising function $\hat{z}_\theta$.

```

1: repeat
2:    $t \sim \text{Uniform}(\{1, 2, \dots, T\}), \epsilon \sim \mathcal{N}(0, I)$ 
3:   Generate predictors  $y_{pre}^0 = B(x_{0:T})$ 
4:   Generate residuals  $y_{Re}^0 = y^0 - y_{pre}^0$ 
5:   Generate latent embedding  $z_\tau^0$  by  $E(y_{Re}^0, y^0)$ 
6:   Generate noised latent  $z_\tau^k = \sqrt{\alpha_k} z_\tau^0 + \sqrt{1 - \alpha_k} \epsilon$ 
7:   Obtain diffusion step  $t$ 's embedding;  $t_\tau^{\text{emb}}$ 
8:   Obtain condition  $\tau_\theta(c)$  by Eq. 13 and Eq. 14
9:   Use the denoising network to generate denoised sample  $z_\tau^0$ 
   by  $\hat{z}_\theta(z_\tau^k, \tau_\theta(c), t_\tau^{\text{emb}})$ 
10:  Calculate the loss  $L_k(\theta)$  by Eq. 16
11: until Converged

```

Algorithm 2 Inference

Input: Trained denoising network $\hat{z}_\theta$, Decoder in pre-trained autoencoder D , Sample $x_{0:T}$ (History), pre-trained predictor backbone B .

Output: Generated corresponding future targets y .

```

1: Initialize  $z_\tau^T \sim \mathcal{N}(0, I)$ 
2: for  $t \leftarrow T$  down to 1 do
3:   Obtain diffusion step  $t$ 's embedding  $t_\tau^{\text{emb}}$ 
4:   Obtain condition  $\tau_\theta(c)$  by Eq. 13 and Eq. 14
5:   if  $k > 1$  then
      $\epsilon \sim \mathcal{N}(0, I)$ 
6:   else
      $\epsilon = 0$ 
7:   end if
8:   Obtain the  $\hat{z}_\tau^0 = \hat{z}_\theta(z_\tau^k, C, t_\tau^{\text{emb}})$ 
9:   Obtain the target  $z_\tau^{t-1}$  by Eq. 17
10: end for
11: Obtain the  $y'_{Re}$  by decoder  $D(z_\tau^0)$ 
12: Obtain the prediction  $y'_{pre}$  by backbone  $B(x_{0:T})$ 
13: Obtain the final result  $y' = y'_{Re} + y'_{pre}$ 
14: return  $y'$ 

```

Diffusion Block Following the design of noisy input, the diffusion model integrates additional information, including noise timesteps t , condition labels c , etc. We explore various transformer blocks that process various inputs differently, ultimately identifying the adaLN-Zero block[6, 29] as the most efficient method. In addition to regressing γ and β , dimension-wise scaling parameters α are also regressed within the DiT block.

Diffusion Inference For each timestep t of the reverse process, a learned denoising distribution p_θ parameterized by θ generates

samples z_{t-1} conditioned on the previously noisier samples z_t , expressed mathematically as follows:

$$z^{t-1} = \hat{\alpha}_t z_t + \hat{\gamma}_t x_\theta(z^t, t | X) + \beta_t \epsilon \quad (17)$$

where the parameters are defined as follows: $\hat{\alpha}_t = \frac{\sqrt{\alpha_t}(1-\alpha_{t-1})}{1-\alpha_t}$, $\hat{\gamma}_t = \frac{\sqrt{\alpha_{t-1}}(1-\alpha_t)}{1-\alpha_t}$, $\beta_t = \frac{(1-\alpha_t)(1-\alpha_{t-1})}{1-\alpha_t}$, and $\epsilon \sim \mathcal{N}(0, 1)$. The detailed derivations are in Appendix A.

Residual Decoder At the conclusion of the final DiT block, the model decodes the residual sequence into the output prediction and combines it with the backbone’s prediction. The decoder projects the predicted $z'_{Re} \in \mathbb{R}^{N \times E}$ into $y'_{Re} \in \mathbb{R}^{N \times L}$ with an autodecoder, which has been pretrained as detailed in Section 3.1. The final output is then computed as:

$$y' = y'_{Re} + y'_{Pre} = \mathcal{D}(z'_{Re}) + y'_{Pre} \quad (18)$$

where y_{Pre} is the prediction made by the pretrained backbone given the input x .

4 Experiments

4.1 Experiment Setup

Dataset: In our experiments, we extensively use seven real-world datasets, including ECL, ETT (four subsets), Traffic, and Weather datasets. Table 1 presents basic statistical information about these datasets.

Table 1: Detailed dataset descriptions. *Dim* denotes the variate number of each dataset. *Frequency* denotes the sampling interval of time point

Dasaset	Dim	Frequency	Information
ETTh1,ETTh2	7	Hourly	Electricity
ETTM1,ETTM2	7	15min	Electricity
ECL	321	Hourly	Electricity
Traffic	862	Hourly	Transportation
Weather	21	10min	Weather

Implementation details: Our experiments contain three steps: Backbone Pretrain, Latent Autoencoder Pretrain, and Diffusion Train and Infer. We give a detailed description of our experiments from the three aspects as below:

- **Backbone Pretrain.** As our framework aims at enhancing forecasting models through residuals, we first train the backbone forecasting models as static indicators according to the open source best performance parameters.
- **Latent Autoencoder Pretrain.** After we excel the residuals, we use autoencoder structure to turn them into latent space and reconstruct. In our autoencoder structure, both the encoder and decoder utilize 3 Transformer layers with 8 heads of the attention. The bridge between the encoder and the decoder is a linear layer, with embedding dimension $m \in \{32, 64, 128, 256\}$ according to variational numbers and predict length of the datasets. The optimization is upon ADAM with an initial learning rate of $5e^{-5}$ and training epochs of 10. We use operations such as cyclic regression loss and limitation of epochs to balance the constraints on

reconstruction and normalization. We will also use calculation formulas such as kurtosis and skewness to verify the standardization of distributions as an auxiliary to the experimental steps. As we give a detailed expression in the AppendixB.

- **Diffusion Train and Infer.** The transformer diffusion adapts to adaLN’s structure with 3 DiT-Blocks. We set the number of timesteps as $T = [50, 100, 150, 200]$, employ square-root noise schedule $\beta^1 = 10^{-3}$ and quad variance schedule $\beta^T = 0.5$. As for the conditional embeddings, we set the look-back windows to 96 for long-term forecasting. We also employ 3 layers of attention with 4 heads to capture condition features. The embedding size is the same as set in the latent autoencoder.

4.2 Main Result

Baselines We carefully choose seven well-acknowledged long-term forecasting models, including (1) Transformer-based methods: Informer[52], Autoformer[46], PatchTST[28], iTransformer[26], and TimeXer[42](2) Linear-based methods: DLinear[48], (3) TCN-based methods: TimesNet[44] as our backbone benchmark. We extensively compare our model with baselines from generative forecasting methods. Including diffusion-based time series models: TimeGrad[31],CSDI[40],TimeDiff[37], promotion-aimed models: GLAFF[22],TMDM[22] which are combined with transformer-based models.

Performance promotion To emphasize our enhancement of multivariate long-term time series forecasting frameworks, we conduct a comparative analysis of prediction results derived from seven datasets using seven backbone models. In Table 2, we establish long-term forecasting frameworks as baseline models, with +*Ours* indicating enhancements based on these frameworks and the best in red. The lower MSE/MAE indicates the more accurate prediction result.

When evaluating our proposed model, we observe strong performance improvements over the baseline models. Notably, we achieve an enhancement of 46.57% on the Informer model, underscoring the significant impact of noise estimation. As the transformer-based models evolve from Informer to Autoformer, PatchTST and iTransformer, we observe a consistent increase in accuracy. Although the degree of improvement from our models decreases within this context, we still achieve notable enhancements across various backbone models. In our comparisons with state-of-the-art models like iTransformer and TimeXer, we document a substantial reduction of 12.67% in MSE, illustrating the vital role of effective estimation as a supplement.

Performance Comparison We evaluate Re-Diffusion with various forecasting models, including diffusion-based models such as CSDI, TimeGard, TimeDiff and TMDM, as well as CNN-based model: TimesNet and the Transformer-based model: iTransformer and GLAFF in Table 3. Re-Diffusion(i) denotes that we employ iTransformer as our backbone. Re-Diffusion achieves remarkable improvement on diffusion-based models as 71.4% promotion TimeGrad, 69.8% promotion on CSDI, 49.5% over TimeDiff in long-term forecasting with the length of 96. These results indicate that Re-Diffusion effectively overcomes the limitations of relying solely on diffusion

Table 2: Performance promotion on seven real-world datasets by applying the proposed framework to long-term forecasting backbones with prediction length $L = 96$, regarding MSE and MAE (lower is better).

Dataset	Electricity		Traffic		Weather		ETTh1		ETTh2		ETTm1		ETTm2	
Method	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Informer	0.350	0.425	0.886	0.506	0.422	0.446	1.02	0.793	1.12	1.151	0.695	0.594	0.798	0.704
+Ours	0.239	0.328	0.675	0.388	0.341	0.324	0.545	0.667	0.876	0.954	0.494	0.503	0.576	0.654
Promotion	31.71%	22.82%	23.81%	23.31%	19.22%	27.04%	46.57%	15.97%	22.32%	17.13%	29.04%	15.38%	27.82%	7.14%
Dlinear	0.196	0.279	0.648	0.396	0.195	0.254	0.386	0.401	0.339	0.394	0.344	0.370	0.189	0.284
+Ours	0.176	0.246	0.547	0.324	0.176	0.213	0.383	0.399	0.327	0.388	0.332	0.365	0.187	0.273
Promotion	10.20%	11.84%	15.62%	18.18%	9.74%	16.14%	0.78%	0.50%	3.54%	1.52%	3.49%	1.35%	1.06%	3.87%
Autoformer	0.185	0.299	0.640	0.402	0.245	0.321	0.437	0.454	0.347	0.394	0.450	0.454	0.212	0.293
+Ours	0.164	0.257	0.504	0.322	0.232	0.307	0.412	0.424	0.321	0.364	0.423	0.403	0.205	0.287
Promotion	11.35%	14.06%	21.25%	19.95%	5.31%	4.37%	5.72%	6.60%	7.56%	7.61%	6.00%	11.23%	3.54%	2.05%
PatchTST	0.165	0.259	0.428	0.281	0.176	0.216	0.392	0.413	0.316	0.361	0.332	0.370	0.196	0.273
+Ours	0.145	0.232	0.398	0.242	0.167	0.203	0.391	0.405	0.312	0.354	0.323	0.362	0.187	0.261
Promotion	12.12%	10.42%	7.01%	13.82%	5.11%	6.02%	0.25%	1.94%	1.27%	1.94%	2.71%	2.16%	4.59%	4.39%
TimeMixer	0.152	0.252	0.501	0.332	0.167	0.215	0.551	0.489	0.422	0.420	0.352	0.383	0.299	0.228
+Ours	0.142	0.229	0.432	0.316	0.154	0.208	0.512	0.432	0.406	0.387	0.338	0.376	0.256	0.221
Promotion	6.58%	9.13%	13.77%	4.82%	7.78%	3.26%	7.08%	11.60%	3.79%	7.86%	3.98%	1.83%	14.48%	3.08%
iTransformer	0.150	0.243	0.416	0.286	0.180	0.218	0.411	0.419	0.323	0.363	0.345	0.375	0.189	0.268
+Ours	0.131	0.228	0.379	0.268	0.165	0.204	0.409	0.413	0.319	0.348	0.319	0.361	0.178	0.257
Promotion	12.67%	6.16%	8.89%	6.29%	8.30%	6.40%	0.49%	1.43%	1.24%	4.13%	7.5%	3.73%	5.81%	4.11%
TimeXer	0.143	0.247	0.458	0.276	0.158	0.206	0.391	0.409	0.290	0.339	0.330	0.365	0.255	0.172
+Ours	0.133	0.227	0.432	0.263	0.154	0.195	0.392	0.406	0.287	0.336	0.304	0.357	0.248	0.169
Promotion	6.99%	8.09%	5.67%	4.71%	2.53%	5.34%	-0.26%	0.73%	1.03%	0.88%	7.88%	2.19%	2.75%	1.74%

Table 3: Multivariate long-term forecasting results on four real-world datasets regarding MSE and MAE.

Dataset	Electricity		Traffic		Weather		ETTm2	
Method	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
TimeGrad	0.65±0.188	0.71±0.149	0.96±0.133	0.81±0.067	0.90±0.129	0.57±0.037	1.36±0.133	0.74±0.113
CSDI	0.56±0.205	0.80±0.156	0.94±0.093	0.67±0.176	0.84±0.073	0.56±0.094	1.28±0.074	0.67±0.064
TimeDiff	0.25±0.024	0.32±0.131	0.68±0.113	0.43±0.052	0.34±0.146	0.37±0.052	0.41±0.014	0.42±0.013
TMDM	0.19±0.007	0.27±0.008	0.60±0.008	0.35±0.009	0.28±0.095	0.25±0.103	0.27±0.023	0.35±0.015
iTransformer	0.15±0.152	0.24±0.132	0.42±0.065	0.29±0.052	0.18±0.104	0.22±0.017	0.19±0.113	0.27±0.014
TimesNet	0.17±0.024	0.27±0.113	0.55±0.038	0.36±0.102	0.18±0.126	0.23±0.064	0.19±0.109	0.28±0.126
GLAFF	0.15±0.096	0.24±0.007	0.43±0.109	0.29±0.044	0.18±0.007	0.22±0.099	0.17±0.033	0.27±0.044
Re-Diffusion(i)	0.13±0.052	0.23±0.051	0.38±0.007	0.26±0.176	0.17±0.095	0.22±0.109	0.18±0.064	0.26±0.064

model for long-term predictions. In the case of TMDM, which utilizes transformer predictions as prior knowledge for diffusion, we adapt to the best results in the paper because their code is inaccessible. For GLAFF, which integrates backbone with timestamps embedding through a linear function, we experiment with its open-source code on iTransformer. Our comparison reveals that Re-Diffusion(i) outperforms them with average 21.2% improvement over TMDM and 7.6% improvement over GLAFF. Based on the same backbone iTransformer, Re-Diffusion(i) outperforms the GLAFF even with static results offered by the pre-trained backbone. We also exhibit the performance of Re-Diffusion on four datasets in Figure 2. The results highlight the effective design of the transformer predictor and its excellent integration with diffusion processes.

Table 4: Results with different conditional styles and the corresponding average relation rate with Residuals. X_{RT} , X_{RE} , X_{RK} , T denotes the residual trends, residual error, residual knowledge and timestamps, respectively. WO denotes without any conditions and the Backbone is iTransformer.

Condition	Electricity		Traffic		Relation
Styles	MSE	MAE	MSE	MAE	Rate
$X_{RT} X_{RE} X_{RK} T$	0.132	0.230	0.379	0.268	0.59
X_{RT}	0.150	0.247	0.407	0.288	0.17
$X_{RT} X_{RE}$	0.141	0.241	0.398	0.278	0.38
$X_{RT} X_{RE} X_{RK}$	0.136	0.234	0.386	0.272	0.49
T	0.147	0.240	0.409	0.277	0.14
WO	0.152	0.250	0.424	0.291	
Backbone	0.150	0.243	0.416	0.286	

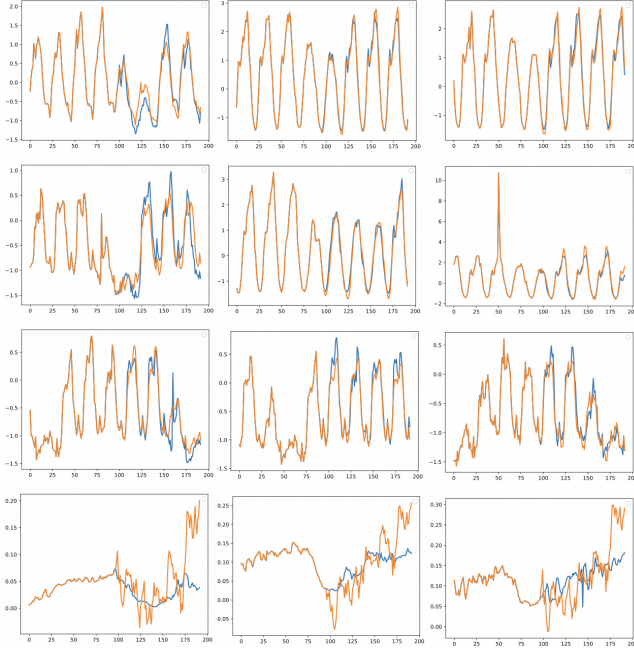


Figure 2: Estimation of the prediction in Electricity, Traffic, Weather and ETTm1 datasets.

4.3 Ablation Study

Residual Autoencoder We compare our residual autoencoder with the autoencoder in LDT[8] in terms of the efficiency for compressing into latent space distribution. In the figure 3, we flatten the data into a one-dimensional vector, and plot the kernel density estimation to visualize the global distribution. The blue line represents the latent space for z , and the green line represents the original space for y . In our autoencoder z is the latent vector of residual data $y = y_{Re}$, with the regularization term $\frac{1}{2}\|z\|^2$ (figures in the first line). In LDT autoencoder, z corresponds to the latent vector of the original data y , with the regularization term KL applied (figures in the second line). Our findings indicate that $y_{Re} \sim \mathcal{N}(0, \sigma^2)$, which supports the validity of our assumptions. Additionally, this demonstrates the existence of the Out-of-Distribution phenomenon. In contrast, the traditional method that utilizes KL with $KL - penalty$ (weight the KL term by a factor like 10^{-8}) strategy as mentioned in LDT, fails to achieve the desired standardization effect in the latent space. These results underscore the importance of effectively leveraging the structure of the original data to compress it into a standardized space. Furthermore, our method offers a viable reference solution for maximizing the potential of Variational Autoencoders (VAEs) in the time-series domain.

Condition Construction As we have demonstrated in our work, we conclude four aspects of information as our conditions, residual trends X_{RT} , residual error X_{RE} , and residual knowledge X_{RK} and TimeStamps T . We concatenate and represent them as Eq. 13, and use the attention mechanism as Eq. 14 to select the best conditional embeddings. We disassemble the conditions in the four directions and combine them separately as condition input for diffusion in Table 4. We also measure their correlation rate with latent residuals

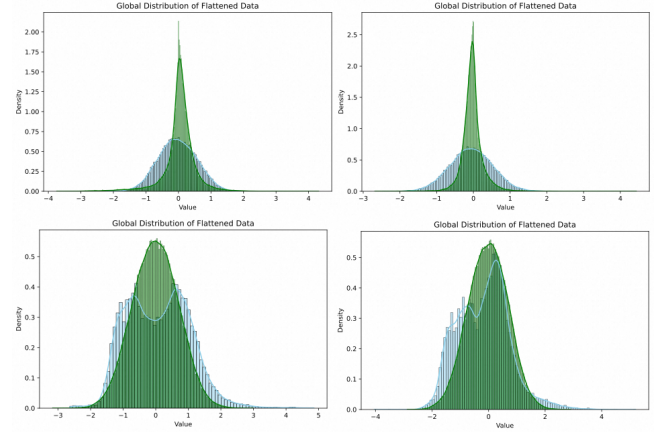


Figure 3: Comparison of original data y and latent z results from our autoencoder (first line) and LDT-encoder[8] (second line). The blue line represents the latent space for z , and the green line represents the original space for y . In our autoencoder, z is the latent vector of residual data $y = y_{Re}$ with the regularization $\frac{1}{2}\|z\|^2$. In LDT autoencoder, z is the latent vector of original data y with the regularization KL .

to support our multiple selection. Timestamps T , whether used independently or in combination, provide conditional information, illustrating the help of extracting timestamp information to improve prediction capabilities. Comparison of **WO** and **Backbone** illustrates the necessity of integrating conditional information for diffusion.

5 Conclusion

In this paper, we propose **Re-Diffusion**, an innovative framework designed for enhancing previous forecasting framework by modeling residuals with diffusion. The optimization function algorithms of previous framework assumes the error residual distribution as Gaussian distribution, and lacks capture of the distribution. However, as latent diffusion model mainly based on compressed Gaussian latent space, which matches the residuals from previous framework naturally. By employing the diffusion, uncertainty distribution can be effectively estimated, which is a supplication to previous forecasting work. Re-Diffusion also stands out as a versatile plug-and-play framework, which can excel on pretrained network results. It also close the gap between point estimates (mostly transformer-based long-term) and distribution based (mostly diffusion-based short-term). Our comprehensive experiments on seven real-world datasets consistently demonstrate Re-Diffusion's superior performance.

6 Acknowledgements

This work was supported by the National Natural Science Foundation of China (Grant Nos. 62202422 and 62372408), and the Zhejiang Province Vanguard Goose-Leading Initiative (2024C03205).

References

- [1] Juan Miguel Lopez Alcaraz and Nils Strodthoff. 2022. Diffusion-based Time Series Imputation and Forecasting with Structured State Space Models. *ArXiv abs/2208.09399* (2022). <https://api.semanticscholar.org/CorpusID:251710482>
- [2] Marin Bilos, Kashif Rasul, Anderson Schneider, Yuriy Nevmyvaka, and Stephan Günnemann. 2022. Modeling Temporal Data as Continuous Functions with Process Diffusion. *ArXiv abs/2211.02590* (2022). <https://api.semanticscholar.org/CorpusID:253370446>
- [3] Hanqun Cao, Cheng Tan, Zhangyang Gao, Yilun Xu, Guangyong Chen, Peng-Ann Heng, and Stan Z. Li. 2022. A Survey on Generative Diffusion Models. *IEEE Transactions on Knowledge and Data Engineering* 36 (2022), 2814–2830. <https://api.semanticscholar.org/CorpusID:265039918>
- [4] Lijuan Cao and Francis Eng Hock Tay. 2001. Financial Forecasting Using Support Vector Machines. *Neural Computing & Applications* 10 (2001), 184–192. <https://api.semanticscholar.org/CorpusID:14475198>
- [5] Prafulla Dhariwal and Alex Nichol. 2021. Diffusion Models Beat GANs on Image Synthesis. *ArXiv abs/2105.05233* (2021). <https://api.semanticscholar.org/CorpusID:234357997>
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ArXiv abs/2010.11929* (2020). <https://api.semanticscholar.org/CorpusID:225039882>
- [7] Hui Fang, Zhu Xiao, Pengfei Zheng, Hongyang Chen, Zhao Li, Jiajun Bu, and Haishuai Wang. 2023. Learning co-occurrence patterns for next destination recommendation. *IEEE Transactions on Mobile Computing* 23, 6 (2023), 7225–7237.
- [8] Shibo Feng, Chunyan Miao, Zhong Zhang, and Peilin Zhao. 2024. Latent Diffusion Transformer for Probabilistic Time Series Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 11 (Mar. 2024), 11979–11987. doi:10.1609/aaai.v38i11.29085
- [9] Hans Fischer. 2011. A History of the Central Limit Theorem: From Classical to Modern Probability Theory. springer:10.1007/978-0-387-87857-7. ISBN 978-0-387-87856-0. MR 2743162. Zbl 1226.60004. [cs.LG]
- [10] Yang Gao, Xiang Zhang, Zhongquan Sun, Payal Chandak, Jiajun Bu, and Haishuai Wang. 2025. Precision adverse drug reactions prediction with heterogeneous graph neural network. *Advanced Science* 12, 4 (2025), 2404671.
- [11] Houxin Gong, Haishuai Wang, Peng Zhang, Sheng Zhou, Hongyang Chen, and Jiajun Bu. 2024. FedMTPP: Federated Multivariate Temporal Point Processes for Distributed Event Sequence Forecasting. *IEEE Transactions on Mobile Computing* (2024).
- [12] Adele Gouttes, Kashif Rasul, Mateusz Koren, Johannes Stephan, and Tofigh Naghibi. 2021. Probabilistic Time Series Forecasting with Implicit Quantile Networks. *ArXiv abs/2107.03743* (2021). <https://api.semanticscholar.org/CorpusID:235765702>
- [13] Albert Gu, Karan Goel, and Christopher R'e. 2021. Efficiently Modeling Long Sequences with Structured State Spaces. *ArXiv abs/2111.00396* (2021). <https://api.semanticscholar.org/CorpusID:240354066>
- [14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. *arXiv:2006.11239 [cs.LG]* <https://arxiv.org/abs/2006.11239>
- [15] John J. Hopfield. 1982. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the United States of America* 79 8 (1982), 2554–8. <https://api.semanticscholar.org/CorpusID:784288>
- [16] Nan Huang, Haishuai Wang, Zihui He, Marinka Zitnik, and Xiang Zhang. 2024. Repurposing foundation model for generalizable medical time series classification. *arXiv preprint arXiv:2410.03794* (2024).
- [17] Yuxin Jia, Youfang Lin, Xinyan Hao, Yan Lin, Shengnan Guo, and Huaiyu Wan. 2023. WITRAN: water-wave information transmission and recurrent acceleration network for long-range time series forecasting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Article 544, 68 pages.
- [18] Xin Jie, Xixi Zhou, Chanfei Su, Zijun Zhou, Yuqing Yuan, Jiajun Bu, and Haishuai Wang. 2024. Disentangled anomaly detection for multivariate time series. In *Companion Proceedings of the ACM Web Conference 2024*. 931–934.
- [19] Diederik P. Kingma and Max Welling. 2013. Auto-Encoding Variational Bayes. *CoRR abs/1312.6114* (2013). <https://api.semanticscholar.org/CorpusID:216078090>
- [20] Marcel Kolloviev, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Wang. 2023. Predict, Refine, Synthesize: Self-Guiding Diffusion Models for Probabilistic Time Series Forecasting. *arXiv:2307.11494 [cs.LG]* <https://arxiv.org/abs/2307.11494>
- [21] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86 (1998), 2278–2324. <https://api.semanticscholar.org/CorpusID:14542261>
- [22] Yuxin Li, Wenchao Chen, Xinyue Hu, Bo Chen, Baolin Sun, and Mingyuan Zhou. 2024. Transformer-Modulated Diffusion Models for Probabilistic Multivariate Time Series Forecasting. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:268880173>
- [23] Y. Li, Xin xin Lu, Yaqing Wang, and De-Yu Dou. 2023. Generative Time Series Forecasting with Diffusion, Denoise, and Disentanglement. *ArXiv abs/2301.03028* (2023). <https://api.semanticscholar.org/CorpusID:255546031>
- [24] Ke Liu, Mengxuan Li, Jiajun Bu, Hongwei Wang, and Haishuai Wang. 2025. CSTSINR: improving temporal continuity via convolutional structured implicit neural representations for time series anomaly detection. *Neural Networks* (2025), 108129.
- [25] Xiao Liu, Yan Li, and Haishuai Wang. 2025. Unraveling Spatial-Temporal and Out-of-Distribution Patterns for Multivariate Time Series Classification. In *Companion Proceedings of the ACM on Web Conference 2025*. 1128–1132.
- [26] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. *arXiv:2310.06625 [cs.LG]* <https://arxiv.org/abs/2310.06625>
- [27] Jielong Lu, Zhihao Wu, Jiajun Yu, Qianqian Shen, Jiajun Bu, and Haishuai Wang. 2025. Where Views Meet Curves: Virtual Anchors for Hyperbolic Multi-View Graph Diffusion. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 2131–2140.
- [28] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2022. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. *ArXiv abs/2211.14730* (2022). <https://api.semanticscholar.org/CorpusID:254044221>
- [29] William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. *arXiv:2212.09748 [cs.CV]* <https://arxiv.org/abs/2212.09748>
- [30] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. 2022. Hierarchical Text-Conditional Image Generation with CLIP Latents. *ArXiv abs/2204.06125* (2022). <https://api.semanticscholar.org/CorpusID:248097655>
- [31] Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. 2021. Autoregressive Denoising Diffusion Models for Multivariate Probabilistic Time Series Forecasting. *arXiv:2101.12072 [cs.LG]* <https://arxiv.org/abs/2101.12072>
- [32] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-Resolution Image Synthesis with Latent Diffusion Models. *arXiv:2112.10752 [cs.CV]* <https://arxiv.org/abs/2112.10752>
- [33] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. 1986. Learning representations by back-propagating errors. *Nature* 323 (1986), 533–536. <https://api.semanticscholar.org/CorpusID:205001834>
- [34] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily L. Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, Seyedeh Sara Mahdavi, Raphael Gontijo Lopes, Tim Salimans, Jonathan Ho, David J. Fleet, and Mohammad Norouzi. 2022. Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding. *ArXiv abs/2205.11487* (2022). <https://api.semanticscholar.org/CorpusID:248986576>
- [35] Chitwan Saharia, Jonathan Ho, William Chan, Tim Salimans, David J. Fleet, and Mohammad Norouzi. 2021. Image Super-Resolution via Iterative Refinement. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (2021), 4713–4726. <https://api.semanticscholar.org/CorpusID:233241040>
- [36] Lifeng Shen, Weiyu Chen, and James T. Kwok. 2024. Multi-Resolution Diffusion Models for Time Series Forecasting. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:271746131>
- [37] Lifeng Shen and James Kwok. 2023. Non-autoregressive Conditional Diffusion Models for Time Series Prediction. *ArXiv abs/2306.05043* (2023). <https://api.semanticscholar.org/CorpusID:259108326>
- [38] Yang Song, Jascha Narain Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2020. Score-Based Generative Modeling through Stochastic Differential Equations. *ArXiv abs/2011.13456* (2020). <https://api.semanticscholar.org/CorpusID:227209335>
- [39] Yan Lindsay Sun, Qifan Song, and Faming Liang. 2021. Consistent Sparse Deep Learning: Theory and Computation. *J. Amer. Statist. Assoc.* 117 (2021), 1981–1995. <https://api.semanticscholar.org/CorpusID:232068722>
- [40] Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. 2021. CSDI: Conditional Score-based Diffusion Models for Probabilistic Time Series Imputation. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:235765577>
- [41] Chengsen Wang, Qi Qi, Jingyu Wang, Haifeng Sun, Zirui Zhuang, Jinming Wu, and Jianxin Liao. 2024. Rethinking the Power of Timestamps for Robust Time Series Forecasting: A Global-Local Fusion Perspective. *arXiv:2409.18696 [cs.LG]* <https://arxiv.org/abs/2409.18696>
- [42] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Yunzhong Qiu, Haoran Zhang, Jianmin Wang, and Mingsheng Long. 2024. TimeXer: Empowering Transformers for Time Series Forecasting with Exogenous Variables. *ArXiv abs/2402.19072* (2024). <https://api.semanticscholar.org/CorpusID:268063560>
- [43] Chenrui Wu, Haishuai Wang, Xiang Zhang, Zhen Fang, and Jiajun Bu. 2024. Spatio-temporal heterogeneous federated learning for time series classification with multi-view orthogonal training. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 2613–2622.
- [44] Haixu Wu, Teng Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2022. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. *ArXiv abs/2210.02186* (2022). <https://api.semanticscholar.org/CorpusID:252715491>

- [45] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:235623791>
- [46] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2022. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. arXiv:2106.13008 [cs.LG] <https://arxiv.org/abs/2106.13008>
- [47] Xinyu Yuan and Yan Qiao. 2024. Diffusion-TS: Interpretable Diffusion for General Time Series Generation. *ArXiv abs/2403.01742* (2024). <https://api.semanticscholar.org/CorpusID:268248137>
- [48] Ailing Zeng, Mu-Hwa Chen, L. Zhang, and Qiang Xu. 2022. Are Transformers Effective for Time Series Forecasting?. In *AAAI Conference on Artificial Intelligence*. <https://api.semanticscholar.org/CorpusID:249097444>
- [49] Chaolv Zeng, Yuan Tian, Guanjie Zheng, and Yunjun Gao. 2024. How Much Can Time-related Features Enhance Time Series Forecasting? arXiv:2412.01557 [cs.LG] <https://arxiv.org/abs/2412.01557>
- [50] Yunhao Zhang and Junchi Yan. 2023. Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting. In *International Conference on Learning Representations*. <https://api.semanticscholar.org/CorpusID:259298223>
- [51] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wan Zhang. 2020. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. *ArXiv abs/2012.07436* (2020). <https://api.semanticscholar.org/CorpusID:229156802>
- [52] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. arXiv:2012.07436 [cs.LG] <https://arxiv.org/abs/2012.07436>

A Derivation of DDPM and DDIM with Direct x_0 Prediction

A.1 Denoising Diffusion Probabilistic Models (DDPM)

Forward Process. Given clean data $x_0 \sim q(x_0)$, the forward process gradually adds Gaussian noise over T steps:

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I), \quad (19)$$

where $\beta_t \in (0, 1)$ is the noise schedule. The marginal distribution at step t is:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I), \quad (20)$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. the noisy latent x_t at step t can be expressed as:

$$x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (21)$$

Reverse Process with x_0 Prediction. Instead of predicting noise ϵ_θ , we parameterize the reverse process to directly predict x_0 . The reverse transition is defined as:

$$p_\theta(x_{t-1}|x_t, c) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, c, t), \Sigma_t), \quad (22)$$

where the mean μ_θ depends on the predicted x_0 :

$$\mu_\theta(x_t, c, t) = \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_\theta(x_t, c, t) + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t. \quad (23)$$

Training Objective. The simplified loss function for training x_θ becomes:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| x_0 - x_\theta \left(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, c, t \right) \right\|^2 \right]. \quad (24)$$

Closed-form x_{t-1} Update. Substituting the predicted $\hat{x}_0 = x_\theta(x_t, c, t)$ into the reparameterized sampling equation:

$$\begin{aligned} x_{t-1} &= \mu_\theta(x_t, c, t) + \sqrt{\beta_t}\epsilon \\ &= \underbrace{\frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}}_{\hat{\gamma}_t} x_\theta(x_t, c, t) + \underbrace{\frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}}_{\hat{\alpha}_t} x_t + \sqrt{\beta_t}\epsilon, \end{aligned} \quad (25)$$

which matches the user's requested form:

$$x_{t-1} = \hat{\alpha}_t x_t + \hat{\gamma}_t x_\theta(x_t, c, t) + \beta_t \epsilon, \quad (26)$$

with coefficients defined as:

$$\hat{\alpha}_t \triangleq \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}, \quad \hat{\gamma}_t \triangleq \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}.$$

A.2 Denoising Diffusion Implicit Models (DDIM)

Generalized Reverse Process. DDIM generalizes the reverse process with a non-Markovian parameterization:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_\theta(x_t, c, t) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2}\epsilon_\theta + \sigma_t\epsilon, \quad (27)$$

where $\epsilon_\theta = \frac{x_t - \sqrt{\bar{\alpha}_t}x_\theta(x_t, c, t)}{\sqrt{1 - \bar{\alpha}_t}}$, and σ_t controls stochasticity.

Deterministic Sampling. When $\sigma_t = 0$, the update becomes deterministic:

$$\begin{aligned} x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1}} \left(\frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_0}{\sqrt{1 - \bar{\alpha}_t}} \right) \\ &= \sqrt{\bar{\alpha}_{t-1}}\hat{x}_0 + \frac{\sqrt{1 - \bar{\alpha}_{t-1}}}{\sqrt{1 - \bar{\alpha}_t}} x_t - \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{\sqrt{1 - \bar{\alpha}_t}} \hat{x}_0 \\ &= \frac{\sqrt{1 - \bar{\alpha}_{t-1}}}{\sqrt{1 - \bar{\alpha}_t}} x_t + \sqrt{\bar{\alpha}_{t-1}} \left(1 - \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{\sqrt{1 - \bar{\alpha}_t}} \right) \hat{x}_0. \end{aligned} \quad (28)$$

Simplifying yields the accelerated update rule:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_\theta(x_t, c, t) + \sqrt{1 - \bar{\alpha}_{t-1}}\epsilon_\theta. \quad (29)$$

B Skewness and Kurtosis of the Gaussian Distribution

We employ skewness and kurtosis as the auxiliary standards of our latent space distribution, which are expected to be normal Gaussian.

Skewness

The skewness measures the asymmetry of a probability distribution. For a random variable X with mean μ and standard deviation σ , the skewness is defined as the standardized third central moment:

$$\text{Skewness} \equiv \gamma_1 = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^3 \right] = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3}. \quad (30)$$

For a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, the skewness is:

$$\gamma_1 = 0. \quad (31)$$

Table 5: Full results of the long-term forecasting task. The input sequence length is set to 96 for all baselines.

Dataset		Electricity		Traffic		Weather		ETTh1		ETTh2		ETTm1		ETTm2	
Method		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Autoformer	96	0.185	0.299	0.640	0.402	0.245	0.321	0.437	0.454	0.347	0.394	0.450	0.454	0.212	0.293
	192	0.208	0.334	0.616	0.382	0.307	0.367	0.500	0.482	0.456	0.452	0.553	0.496	0.281	0.340
	336	0.231	0.338	0.622	0.337	0.359	0.395	0.521	0.496	0.482	0.486	0.621	0.537	0.339	0.372
	720	0.254	0.361	0.660	0.408	0.419	0.428	0.514	0.512	0.515	0.511	0.671	0.561	0.433	0.432
Dlinear	96	0.196	0.279	0.648	0.396	0.195	0.254	0.386	0.401	0.339	0.394	0.344	0.370	0.189	0.284
	192	0.196	0.285	0.598	0.370	0.237	0.296	0.437	0.432	0.477	0.476	0.380	0.389	0.284	0.362
	336	0.209	0.301	0.605	0.373	0.283	0.335	0.481	0.459	0.594	0.541	0.413	0.413	0.369	0.427
	720	0.245	0.333	0.645	0.394	0.345	0.381	0.519	0.516	0.831	0.657	0.474	0.453	0.554	0.522
TimesNet	96	0.171	0.274	0.546	0.364	0.180	0.220	0.384	0.402	0.340	0.374	0.338	0.375	0.187	0.267
	192	0.184	0.289	0.617	0.366	0.219	0.261	0.436	0.429	0.402	0.414	0.374	0.387	0.249	0.309
	336	0.198	0.300	0.629	0.366	0.280	0.306	0.491	0.469	0.452	0.452	0.410	0.411	0.321	0.351
	720	0.220	0.320	0.640	0.373	0.365	0.356	0.521	0.500	0.462	0.462	0.478	0.450	0.408	0.403
PatchTST	96	0.165	0.259	0.428	0.281	0.176	0.216	0.392	0.413	0.316	0.361	0.332	0.370	0.196	0.273
	192	0.199	0.269	0.466	0.296	0.225	0.259	0.460	0.445	0.388	0.400	0.367	0.385	0.241	0.302
	336	0.215	0.295	0.482	0.304	0.278	0.297	0.501	0.466	0.426	0.433	0.399	0.410	0.305	0.343
	720	0.256	0.327	0.514	0.322	0.354	0.348	0.500	0.488	0.431	0.446	0.454	0.439	0.407	0.398
iTransformer	96	0.150	0.243	0.416	0.286	0.180	0.218	0.411	0.419	0.323	0.363	0.345	0.375	0.189	0.268
	192	0.162	0.253	0.417	0.287	0.221	0.254	0.441	0.436	0.380	0.400	0.387	0.426	0.250	0.309
	336	0.178	0.269	0.433	0.283	0.278	0.296	0.487	0.458	0.428	0.432	0.426	0.420	0.311	0.348
	720	0.225	0.317	0.467	0.302	0.358	0.349	0.503	0.491	0.427	0.463	0.491	0.456	0.412	0.407
TimeXer	96	0.143	0.247	0.458	0.276	0.158	0.206	0.391	0.409	0.290	0.339	0.330	0.365	0.172	0.255
	192	0.157	0.256	0.453	0.282	0.204	0.247	0.429	0.435	0.363	0.389	0.362	0.383	0.237	0.299
	336	0.176	0.275	0.473	0.289	0.261	0.290	0.468	0.448	0.414	0.423	0.395	0.407	0.296	0.338
	720	0.211	0.306	0.516	0.307	0.340	0.341	0.469	0.461	0.408	0.432	0.452	0.441	0.392	0.394
Re-Diffusion (i)	96	0.131	0.228	0.379	0.268	0.165	0.204	0.409	0.413	0.319	0.348	0.319	0.361	0.178	0.252
	192	0.147	0.244	0.404	0.279	0.221	0.249	0.439	0.431	0.376	0.401	0.385	0.425	0.248	0.301
	336	0.165	0.253	0.428	0.283	0.272	0.291	0.481	0.452	0.423	0.426	0.419	0.418	0.310	0.339
	720	0.213	0.302	0.451	0.298	0.342	0.342	0.501	0.484	0.421	0.457	0.478	0.449	0.408	0.401
Re-Diffusion (T)	96	0.133	0.227	0.432	0.263	0.154	0.195	0.392	0.406	0.287	0.336	0.304	0.357	0.248	0.169
	192	0.144	0.251	0.450	0.281	0.199	0.244	0.431	0.429	0.361	0.384	0.354	0.381	0.233	0.294
	336	0.171	0.269	0.469	0.285	0.258	0.288	0.462	0.443	0.415	0.420	0.396	0.401	0.291	0.332
	720	0.209	0.297	0.501	0.301	0.337	0.339	0.464	0.457	0.401	0.430	0.448	0.439	0.386	0.397

Kurtosis

The kurtosis quantifies the "tailedness" of a distribution. It is defined as the standardized fourth central moment. For a random variable X :

$$\text{Kurtosis} \equiv \beta_2 = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^4 \right] = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4}. \quad (32)$$

For a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, the kurtosis is:

$$\beta_2 = 3. \quad (33)$$

C Full Forecasting Results

The full multivariate forecasting results are provided in the following section due to the space limitation of the main text. We extensively evaluate competitive counterparts on long-term forecasting tasks in Table 5. It contains the detailed results of all prediction lengths of well-acknowledged forecasting benchmarks, with the best results among all methods represented in **red** and the best results among backbones represented in **bold**. Re-Diffusion(i) and Re-Diffusion(T) denotes the different backbone with iTransformer and TimeXer.